

Operating Systems: Homework 2 (HW2)

Instructor: Dr. Probir Roy

Due Time: 11:59PM, 10/18/2023

Instructions

- **No teamwork** is allowed for all homework assignment.
- VeriCite option has been enabled on Canvas to compare your submission with other submissions from this semester and past semesters and Internet database.
- Your solution should be written in a “.docx”, “.doc”, or “.pdf” format and submitted to the assignment folder HW2 on Canvas.

Questions

1. What are included in the address space of a process?
2. How does a base-and-bounds dynamic memory relocation scheme achieve memory protection?
3. We use simulator `segmentation.py` to study how virtual-to-physical address translation works with segmentation.¹ We ran a configuration and got the following output with a slight modification. Please fill in the blanks.

ARG address space size 1k

ARG phys mem size 64k

Segment register information:

Segment 0 base (grows positive) : 0x00000cef (decimal 3311)

Segment 0 limit : 323

Segment 1 base (grows negative) : 0x00006c1f (decimal 27679)

Segment 1 limit : 409

Virtual Address Trace

VA 0: 0x00000233 (decimal: 563) --> []

¹All simulators and READMEs in homework assignments can be found on this page: <http://pages.cs.wisc.edu/~remzi/OSTEP/Homework/homework.html>. They are also stored in the Docker image from P1 under Folder /simulation/HW-MLFQ with Python 2.7 support.

```

VA  1: 0x0000013c (decimal:  316) --> [           ]
VA  2: 0x000003ac (decimal:  940) --> [           ]

```

For each virtual address, either write down the physical address it translates to OR write down that it is an out-of-bounds address (a segmentation violation). For this problem, you should assume a simple address space with two segments: the top bit of the virtual address can thus be used to check whether the virtual address is in segment 0 (topbit=0) or segment 1 (topbit=1). Note that the base/limit pairs given to you grow in different directions, depending on the segment, i.e., segment 0 grows in the positive direction, whereas segment 1 in the negative.

Write the steps on how you manually obtain the final values.

4. Given separated memory holes of 210KB, 1070KB, 350KB, 1360KB, and 550KB (in order), how would each of the following contiguous memory allocation policies place segments of 960KB, 440KB, 240KB, and 770KB of a process (in order): first-fit, next-fit, best-fit, and worst-fit?
5. (1) Does paging have any internal fragmentation? If it does not, why? If it does, how can we reduce such fragmentation? (2) Does paging have any external fragmentation? If it does not, why? If it does, how can we reduce such fragmentation?
6. We use Simulator `paging-linear-translate.py` to study how the address translation work with the linear paging table. We ran a configuration and got the following output with a slight modification. Please fill in the blanks.

```

ARG address space size 16k
ARG phys mem size 512k
ARG page size 2k
ARG verbose True
ARG addresses -1

```

The format of the page table is simple:

The high-order (left-most) bit is the VALID bit.

If the bit is 1, the rest of the entry is the PFN.

If the bit is 0, the page is not valid.

Use verbose mode (-v) if you want to print the VPN # by each entry of the page table.

Page Table (from entry 0 down to the max size)

```

[      0]  0x00000000
[      1]  0x8000000c
[      2]  0x80000033
[      3]  0x800000e5
[      4]  0x00000000
[      5]  0x800000f8
[      6]  0x800000eb
[      7]  0x00000000

```

Virtual Address Trace

```

VA 0: 0x00002512 (decimal:      9490) --> [          ]
VA 1: 0x00001956 (decimal:      6486) --> [          ]
VA 2: 0x00003076 (decimal:     12406) --> [          ]

```

For each virtual address, write down the physical address it translates to OR write down that it is an out-of-bounds address.

Write the steps on how you manually obtain the final values.

7. Consider a paging system with the page table stored in memory. 80 percent of all page-table references are found in the TLBs. We assume that a memory reference takes 300 ns and finding a page-table entry in the TLBs takes 50 ns. What is the effective access time?
9. How could segmentation be used to reduce the page table size? How could multiple-level paging be used to reduce the page table size?
10. Consider a demand-paging system with a paging disk that has an average access/transfer time of 50 ms. Addresses are translated through a page table in main memory, with an access time of 500 ns per memory access. Thus, each memory reference through the page table takes two accesses. To improve this time, we have added a TLB that reduces access time to one memory reference if the page-table entry is in the TLB. Assume that 80% of the accesses are in the TLB and that, of those remaining, 15% (or 3% of the total) cause page faults. We assume that the TLB access time is 20 ns. What is the effective access time?
11. Assume a physical memory of 4 page frames and initially all frames are empty. Given the reference string **5, 0, 5, 4, 2, 1, 2, 4, 5, 1, 2, 1, 4, 0, 1**, show the page replacement procedures and give the number of page faults for the following page replacement policies: (a) FIFO, (b) OPT, (c) LRU, and (d) Second Chance.